

1. What is Informed Search?

- Informed search uses domain knowledge in the form of heuristics to guide the search toward a goal.
- A heuristic function $h(n)$ estimates the distance or cost from a state n to a nearest goal state.
- It provides additional information beyond the problem definition, helping to explore more promising states first.

Key Idea: A heuristic estimates how close a state is to a goal. It guides the search to explore fewer, more promising states.

Example: In route finding, $h(n)$ can be the straight-line distance to the destination.

3. A* Search

- Uses both the path cost and heuristic estimate.
- Evaluation function: $f(n) = g(n) + h(n)$
- $g(n)$: cost from the initial state to state n (actual path cost so far).
- $h(n)$: estimated cost from state n to a goal (heuristic estimate).
- A* balances the actual cost and estimated cost, expanding states with the lowest estimated total cost.
- If $h(n)$ is admissible (never overestimates the true cost), A* is complete and optimal.

Properties (A*):

- Complete (if h is admissible).
- Optimal (if h is admissible).
- Often more efficient than uninformed search.

2. Greedy Best-First Search (GBFS)

- Uses only the heuristic estimate to choose the next state.
- Evaluation function: $f(n) = h(n)$
- Expands the node that appears closest to the goal according to $h(n)$.
- Usually fast and explores fewer nodes.
- Not complete (can get stuck in local minima).
- Not optimal (may find a suboptimal solution).
- Useful when a quick, approximate solution is acceptable.

Idea:

Always expand the node with the smallest $h(n)$ (appears closest to the goal).

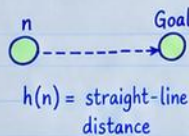
4. Uninformed vs. Informed Search

Uninformed Search	Informed Search
Uses only problem definition (no extra knowledge).	Uses heuristic function $h(n)$ to guide search.
Explores states systematically (e.g., BFS, DFS, UCS).	Explores more promising states first (e.g., GBFS, A*).
Often explores many irrelevant states.	Usually explores fewer states.
No domain-specific knowledge.	Uses domain knowledge through heuristics.
Examples: BFS, DFS, UCS, IDS, DLS.	Examples: Greedy Best-First Search, A* Search.

5. Example Heuristics

(a) Straight-Line Distance (Euclidean Distance)

- Used in route finding problems.
- $h(n)$ = straight-line distance from n to goal.
- Easy to compute and often effective.



(b) Misplaced Tiles (8-Puzzle)

- Counts the number of tiles not in their goal position.
- $h(n)$ = number of misplaced tiles.
- Simple heuristic but less accurate than Manhattan distance.

State n	Goal state
1 2 3	1 2 3
4 5 6	4 5 6
7 0 8	7 8 0

$h(n) = 1$ (tile 8 misplaced)

(c) Manhattan Distance (8-Puzzle)

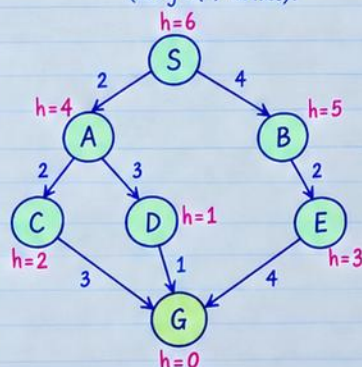
- Sum of horizontal and vertical distances of each tile from its goal position.
- $h(n) = \sum (|x_i - x_i^*| + |y_i - y_i^*|)$ (for each tile i , excluding blank)
- More informed (better estimate) than misplaced tiles.

1	2	3
4	0	6
7	5	8

$h(n) = 2$
(tile 5 is 2 moves away)

6. Heuristic-Guided Search Example

- The heuristic helps the search explore states that appear closer to the goal.
- Example: Best-First or A* search on a small graph (using $h(n)$ values).



How search proceeds (e.g., A* or GBFS):

1. Start at S.
2. Use $h(n)$ (and $g(n)$ for A*) to choose next node.
3. Expand the most promising node (e.g., A with $h=4$).
4. Continue until goal G is reached.

7. Key Takeaways

1. Informed search uses heuristics to guide the search toward a goal.
2. Heuristic $h(n)$ estimates the distance or cost to a goal.
3. Greedy Best-First Search: $f(n) = h(n)$; fast but not optimal.
4. A* Search: $f(n) = g(n) + h(n)$; complete and optimal if h is admissible.
5. $g(n)$ is the actual cost from the start, $h(n)$ is the estimated cost to the goal.
6. Informed search usually explores fewer states than uninformed search.
7. Common heuristics: straight-line distance, misplaced tiles, Manhattan distance.
8. Good heuristics make search more efficient and effective.

1. Admissible Heuristics

- A heuristic $h(n)$ is admissible if it never overestimates the true remaining cost $h^*(n)$ from n to a goal.
- Admissible heuristics are optimistic lower bounds on the actual cost.
- They can be derived from relaxed problems or background knowledge about the problem.
- Examples: straight-line distance (route finding), misplaced tiles (8-puzzle).

Key Idea: If h is admissible, A^* tree search is optimal.

2. Consistent (Monotone) Heuristics

- A heuristic $h(n)$ is consistent if for every action a and successor n' , $h(n) \leq c(n, a, n') + h(n')$.
- This is the triangle inequality: the estimated cost from n to a goal is at most the step cost plus the estimated cost from the successor.
- f -values never decrease along a path: $f(n') \geq f(n)$.
- Consistency implies admissibility.
- With a consistent heuristic, A^* graph search is optimal (and does not need to reopen nodes).

Key Idea: If h is consistent, A^* graph search is optimal.

3. Why A^* Works

- A^* chooses the node with the smallest $f(n) = g(n) + h(n)$.
- $g(n)$ is the exact cost from the start to n (known path cost).
- $h(n)$ estimates the remaining cost from n to a goal.
- If h is admissible, no cheaper solution can be missed, because any node on an optimal path will have $f(n) \leq C^*$ (cost of the optimal solution).

$$A^* = \underbrace{\text{actual cost so far}}_{g(n)} + \underbrace{\text{estimated future cost}}_{h(n)}$$

4. Heuristic Dominance

- If $h_2(n) \geq h_1(n)$ for all n , and both heuristics are admissible, then h_2 dominates h_1 .
- A dominating heuristic is at least as informed and usually expands fewer nodes.
- Stronger heuristics give better guidance to the search.
- Example: In the 8-puzzle, Manhattan distance dominates misplaced tiles (both are admissible).

Comparison (8-puzzle)

Heuristic	Estimate $h(n)$	Nodes Expanded (typical)	Informative?
Misplaced tiles	# of misplaced tiles	More	Weaker
Manhattan distance	Sum of tile distances	Fewer	Stronger (dominates)

5. How to Build Heuristics

- Relaxed problems: solve a simplified version of the problem (fewer constraints) to get a lower bound.
- Solving subproblems: break the problem into parts and combine their costs.
- Pattern databases: precompute exact costs for a subset of variables (e.g., a subset of tiles in the 8-puzzle).
- Using domain knowledge: exploit problem structure (e.g., straight-line distance, Manhattan distance).
- A good heuristic should be informative (close to h^*) but cheap to compute.

6. Example: 8-Puzzle Heuristics

Initial State (n)

2	8	3
1	6	4
7		5

Goal State

1	2	3
8		4
7	6	5

Both heuristics are admissible, and Manhattan distance dominates misplaced tiles ($h_2(n) \geq h_1(n)$).

Heuristic values for n :

Misplaced tiles $h_1(n)$:

- 5 tiles are misplaced $\Rightarrow h_1(n) = 5$

Manhattan distance $h_2(n)$:

- Sum of tile distances (excluding blank) $\Rightarrow h_2(n) = 10$

7. Key Takeaways

- Admissible heuristic: $h(n)$ never overestimates the true remaining cost $h^*(n)$; provides an optimistic lower bound.
- Consistent heuristic: $h(n) \leq c(n, a, n') + h(n')$; f -values never decrease and consistency implies admissibility.
- A^* tree search is optimal with an admissible heuristic; A^* graph search is optimal with a consistent heuristic.
- Heuristic dominance: if $h_2(n) \geq h_1(n)$ (both admissible), h_2 is at least as informed and expands fewer nodes.
- Heuristics can be built using relaxed problems, subproblems, pattern databases, or domain knowledge.
- Good heuristics are informative (close to h^*) but cheap to compute, making search more efficient and effective.

1. Local Search

- Operates on complete states rather than paths.
- Useful for large optimization problems (e.g., scheduling, machine learning, circuit design).
- Often uses little memory.
- Goal is to find a good state (ideally optimal) rather than an explicit path to the goal.

Key Idea: Keep a single (or small set of) current state(s) and move to better states in the state space.

2. Hill Climbing

- Start with an initial state and move to a better neighboring state (according to an evaluation function).
- Variants:
 - Steepest-ascent hill climbing: choose the best neighbor.
 - First-choice hill climbing: choose the first better neighbor.
 - Stochastic hill climbing: choose a better neighbor at random (often with probability proportional to improvement).

Advantages:

- Simple and easy to implement.
- Uses very little memory.
- Often finds a good solution quickly.

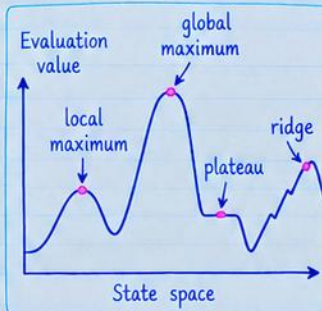
Disadvantages:

- Can get stuck in local maxima, plateaus, or ridges.
- Complete and optimal? No.
- Sensitive to the initial state.

3. Common Problems

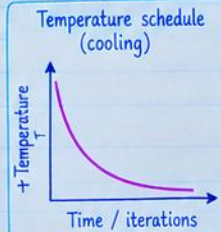
- Local maxima: a state better than all neighbors, but not the global optimum.
- Plateaus: flat regions with many states of the same value (no better neighbor).
- Ridges: narrow paths to higher values; requires non-greedy moves.
- Hill climbing can get stuck in any of these.
- Random-restart hill climbing runs hill climbing from many random initial states to reduce the chance of getting stuck.

Hill climbing is simple but not guaranteed to find the global optimum.



4. Simulated Annealing

- Sometimes accepts worse moves (moves to lower-valued states) with a certain probability.
- Temperature parameter controls the level of randomness.
- High temperature = more random moves (exploration).
- Low temperature = fewer random moves (exploitation).
- Can escape local maxima and is more likely to find a global optimum than simple hill climbing.



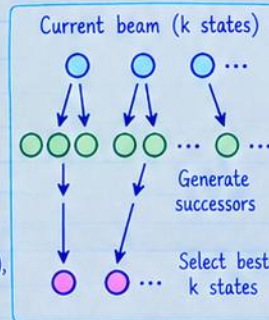
Typical schedule:

$T_{t+1} = \alpha T_t$ ($0 < \alpha < 1$)
High T: explore
Low T: exploit

Idea: Gradually decrease the temperature (cooling schedule) over time.

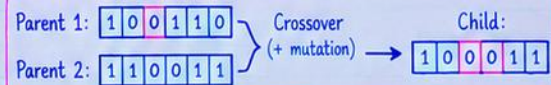
5. Local Beam Search

- Keep k states at once (a beam of states).
- Generate all successors of the k states.
- Select the best k states for the next iteration.
- Stochastic beam search: select k states at random from the successors, with probability based on their evaluation (helps preserve diversity).
- Can escape some local maxima (better than hill climbing), but still not guaranteed to find the global optimum.



6. Genetic Algorithms

- Maintain a population of candidate solutions (states).
- Use a fitness function to evaluate solutions.
- Repeat: selection, crossover, and mutation to create a new population.
- Useful for large and complex search spaces where other methods may get stuck.



7. Comparison

Aspect	Hill Climbing	Simulated Annealing	Local Beam Search	Genetic Algorithms
Memory usage	Very low (1 state)	Very low (1 state)	Low (k states)	Moderate (population)
Escape local maxima?	No (can get stuck)	Yes (with probability)	Sometimes (better than HC)	Yes (via diversity)
Main idea	Move to a better neighbor	Occasionally accept worse moves	Keep k best states	Evolve population (selection, crossover, mutation)

8. Key Takeaways

- Local search operates on complete states and uses little memory.
- Hill climbing is simple and fast but can get stuck in local maxima, plateaus, or ridges.
- Simulated annealing accepts worse moves and uses a cooling schedule to escape local maxima.
- Local beam search keeps multiple states to maintain diversity.
- Genetic algorithms use a population with selection, crossover, and mutation.
- These methods are useful for large optimization problems and often find good (though not guaranteed optimal) solutions.

5. Exploration discovers new states, while exploitation follows currently good options.
6. LRTA* updates heuristic estimates while acting, combining search with learning.
7. Memory-bounded algorithms (IDA*, RBFS, SMA*) trade additional re-expansion for lower space usage.